Chapitre 0 — Avant tout | Partie B — Le temps | leçon 0.16.3 sur 153 | difficulté 0 / 10

0.16.3. La fonction effacer() — effacer des cases

#include <effacer> : effacer(colonne, ligne, quoi) vide des cases ; effacer() sans rien vide tout l'écran.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `effacer(colonne, ligne, quoi)` remet des cases **à vide**. « quoi » dit combien : un **nombre de cases**, ou le **texte lui-même** — `effacer(4, 8, "BON")` efface trois cases, le compilateur compte les lettres. Sans argument, `effacer()` vide **tout** l'écran.

La ligne à écrire : `#include <effacer>`.

Ses arguments : la colonne, la ligne, puis le nombre de cases ou le texte.

Ce qu'elle coûte : environ **120 octets** la première fois.

Essaie : remplace `3` par `"BONJ"` : quatre cases partent.

On retrouve effacer dans 95 leçons

0.17, 0.17.1, 0.17.2, 0.33, 0.33.1, 0.33.2, 0.65, 0.65.1, 0.65.2, 0.66, 0.66.1, 0.66.2, 0.67, 0.67.1, 0.67.2, 0.68, 0.68.1, 0.68.2, 0.69, 0.69.1, 0.69.2, 0.76.5, 0.76.6, 0.78, 0.78.1, 0.82, 0.82.1, 0.83.2, 0.84, 0.90.1, 0.90.2, 0.90.4, 0.90.5, 0.90.6, 0.90.7, 0.90.8, 0.91, 0.91.1, 0.91.2, 0.91.3, 0.91.4, 0.91.5, 0.91.6, 0.91.7, 0.91.8, 0.91.9, 0.92, 0.92.1, 0.92.2, 0.93.1, 0.93.2, 0.93.3, 0.93.4, 0.93.5, 0.93.6, 0.93.7, 0.93.8, 0.93.9, 0.94, 0.94.1, 0.95, 0.95.1, 0.96, 0.96.1, 0.97, 0.97.1, 0.98, 0.98.1, 0.99, 0.99.1, 0.100, 0.100.1, 0.101, 0.101.1, 0.102, 0.102.1, 0.103, 0.104, 0.105, 0.105.1, 0.106, 0.107, 0.107.1, 0.108, 1.2, 1.4, 1.6, 1.9, 1.11, 1.12, 2.3, 8, 16, 126, 134

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **chaîne de caractères (texte)** — Des lettres entre guillemets, "COMME CECI" : un texte pris tel quel. La police de la console n'a que des majuscules, sans accents.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **coordonnées** — Deux nombres qui disent où : la colonne (vers la droite) puis la ligne (vers le bas), à partir de (0, 0) en haut à gauche.
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **texte()** — Écrit des lettres à l'écran, sur la grille de 20 colonnes et 18 lignes : texte(colonne, ligne, "MOT").
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`// ---- #include <effacer> : effacer des cases ----`	Un commentaire : une phrase pour celui qui lit (« ---- #include <effacer> : effacer des cases ---- »). La console l'ignore.
`// On écrit BONJOUR, puis on efface ses trois premières cases : il reste JOUR.`	Un commentaire : une phrase pour celui qui lit (« On écrit BONJOUR, puis on efface ses trois premières cases : il reste JOUR. »). La console l'ignore.
`#include <texte>  // pour écrire d'abord le mot`	Demande à la console « texte » (ce que c'est : écrit un texte à l'écran, à la taille qu'on veut). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « pour écrire d'abord le mot ».)
`#include <effacer>  // effacer() : remettre des cases à vide`	Demande à la console « effacer » (ce que c'est : efface des cases, ou tout le fond). Sans cette ligne, le compilateur refuserait de

	l'employer. (Le commentaire le dit : « effacer() : remettre des cases à vide ».)
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.
`texte(4, 8, "BONJOUR");`	Appel de la fonction texte de la console, avec les arguments 4, 8, "BONJOUR". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT). Un 4e argument est une TAILLE, sans limite : 1 normal, 2 deux fois plus grand, 3… — texte(2, 2, "SUPER", 3). Avec une taille, le texte et la taille s'écrivent en clair ; ce qui dépasse de l'écran est coupé.
`effacer(4, 8, 3);  // trois cases à partir de (4, 8) : B, O et N`	Appel de la fonction effacer de la console, avec les arguments 4, 8, 3. Ce qu'elle fait : efface un texte à l'écran : « quoi » est le texte lui-même, ou un nombre de cases. Avec un Mot : effacer(SALUT). (Le commentaire le dit : « trois cases à partir de (4, 8) : B, O et N ».)
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().
`image();`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu.
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **345 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage        49 o  ← la console, à l'allumage  — la mise en route de la console
  main()           26 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage    28 o  ← image()  — attend l'image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l'écran soit libre
  AttendreAcces    12 o  ← EffacerCases, EcrireTexte  — attend de pouvoir écrire une case
  EffacerCases     12 o  ← effacer()  — efface des cases
  EcrireTexte      10 o  ← texte()  — écrit un texte, lettre par lettre
  CopierTuiles     67 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles           96 o  ← CopierTuiles  — 6 lettres sur 43
  "BONJOUR"         7 o  ← texte()  — un texte gravé
Lettres gravées : BJNORU
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EcrireNombre, Diviser, Reste, Multiplier,
DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette,
DonneesTransfert, Notes
Fonctions du langage appelées :
  texte()   × 1    11 o sur place, dans main()  + EcrireTexte (10 o)
  effacer() × 1     8 o sur place, dans main()  + EffacerCases (12 o)
  image()   × 1     3 o sur place, dans main()  + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <effacer> : effacer des cases ----
// On écrit BONJOUR, puis on efface ses trois premières cases : il reste JOUR.

#include <texte>     // pour écrire d'abord le mot
#include <effacer>   // effacer() : remettre des cases à vide
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
  texte(4, 8, "BONJOUR");
  effacer(4, 8, 3);         // trois cases à partir de (4, 8) : B, O et N

  while (true) {
    image();
  }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

> Trois cases vides, puis JOUR.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ les trois premières cases sont vides

✓ JOUR reste (« JOUR »)

Ensuite : **0.17. Une lettre qui avance de 5 cases** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++